MACHINE LEARNING E MALATTIE CARDIOVASCOLARI: QUANDO STATISTICA E MEDICINA LAVORANO INSIEME.

Mattia Toffolo, matricola n. 820438*
Progetto per l'Esame di Machine Learning, 15/07/2021

In questo articolo scientifico ho svolto un'analisi statistica su dati clinici raccolti su un insieme di pazienti affetti da insufficienza cardiaca. L'obbiettivo è fare previsione sulla sopravvivenza degli individui e individuare quali sono le variabili piú influenti. A tale scopo ho addestrato piú algoritmi provenienti da diverse aree del Machine Learning: un modello statistico logit, una random forest, un albero di classificazione, una support vector machine lineare, una support vector machine gaussiana e un k nearest neighbor. L'algoritmo piú performante è risultato essere il random forest. Mentre, tra le esplicative, le piú importanti sono la percentuale di sangue rilasciato dal cuore in ogni contrazione e il livello di creatinina. Infine, su queste due variabili, ho addestrato un'altra random forest, che è risultata parsimoniosa nel numero di attributi utilizzati e piú accurata nelle previsioni. Come principale criticità dello studio sottolineo le poche osservazioni presenti nel dataset.

Nomenclature

ML	=	machine learning
TRS	=	training set
VS	=	validation set
TS	=	test set
CV	=	cross validation
AUC	=	area under the curve
GLM	=	generalized linear model
RF	=	random forest
OOB	=	out of bag
DC	=	decision tree
SVM	=	support vector machine
$GSVM$	=	gaussian support vector machine
KNN	=	k nearest neighbor

I. Introduction

Le malattie cardiovascolari sono scompensi del cuore e delle vene connesse. Esse, limitando il pompaggio di sangue, impediscono al cuore di soddisfare la richiesta del corpo, causando così molte patologie e problemi di salute, tra cui infarti, ictus e, nei casi peggiori, la morte. Dato il ruolo protagonista di un organo come il cuore, la previsione dell'insufficienza cardiaca è una priorità per medici e ricercatori, anche se, ad oggi, non esiste ancora un metodo coerente ed affidabile per classificare un individuo come completamente privo di rischio o soggetto a pericolo. In questo contesto, i dati delle cartelle cliniche dei pazienti possono rivelarsi un'utile risorsa informativa per analizzare le relazioni tra determinati eventi, caratteristiche chiave individuali e la presenza di malattie cardiovascolari. L'apprendimento meccanico applicato a questi dati può rivelarsi un vantaggioso strumento, sia per prevedere la sopravvivenza delle unità statistiche (i.e. i pazienti) sia per cercare caratteristiche cliniche che possono provocare scompensi cardiaci. [1][2]

Al fine di dimostrare le potenzialità di quanto appreso durante il corso di Machine Learning, in questo articolo

*Studente di Scienze Statistiche ed Economiche, Dipartimento di Economia, Metodi Quantitativi e Strategie di Impresa, m.toffolo1@campus.unimib.it.

scientifico, propongo un'analisi statistica avanzata e multivariata sull'insufficienza cardiaca di un certo insieme di pazienti. Tale analisi ha l'obiettivo di fornire, in fase di pre screening, un supporto decisionale al consulto dei medici, specialisti condizionati dal fattore umano.

In particolare, descrivo il dataset nella sua interezza, i software utilizzati e, giustificando le scelte eseguite, i metodi impiegati per l'analisi, nel paragrafo II. Nella sezione III riporto il prodotto della ricerca, cioè le prestazioni e le performance degli algoritmi addestrati, e la discussione degli stessi. Infine, nel paragrafo IV, riporto alcune conclusioni in merito al lavoro svolto.

II. Materials and methods

Per quest'analisi si dispone di una banca dati *heart_failure_clinical_records_dataset.csv*, raccolta presso l'istituto di cardiologia dell'Allied Hospital di Faisalabad (Punjab, Pakistan), nel periodo che va da aprile a dicembre del 2015. Il file è di tipo cross-section, contiene un'istanza per ogni paziente, tutti con precedenti problemi di salute cardiaca. Esso è composto da 299 righe e 13 variabili cliniche, che riportano informazioni generali, sanitarie e sullo stile di vita personale. Gli attributi sono: "age" (l'età del paziente espressa in anni), "anaemia" (valore booleano, se il paziente ha avuto una diminuzione di globuli rossi o di emoglobina), "high blood pressure" (valore booleano, se il paziente soffre di ipertensione), "creatinine phosphokinase" (livello dell'enzima CPK nel sangue misurato in mcg/L), "diabetes" (valore booleano, se il paziente soffre di diabete), "ejection fraction" (percentuale di sangue rilasciato dal cuore in ogni contrazione), "platelets" (piastrine nel sangue misurate il kilopiastrine/mL), "sex" (valore binario, il sesso del paziente), "serum creatinine" (livello di creatinina nel sangue misurata in mg/dL), "serum sodium" (livello di sodio nel sangue misurato in mEq/L), "smoking" (valore booleano, se il paziente è fumatore) e "time" (periodo di follow-up, dai 4 ai 285 giorni). La variabile target è "death event" ed assume il valore 1 se il paziente è morto durante il periodo di follow-up, 0 altrimenti. Data la precisione e l'efficienza degli strumenti medici avanguardisti, non mi aspetto rumore ambientale nelle rilevazioni. Il dataset è in formato CSV* ed è disponibile presso la fonte web UCI Machine Learning Repository[3]. É una fonte statistica esterna e statuaria, è divulgata gratuitamente ed è accessibile a chiunque.

Per svolgere il corpo principale dell'analisi mi sono servito dell'ambiente di sviluppo integrato RStudio, il quale permette di programmare nel linguaggio statistico R. Per prendere dimestichezza con un nuovo strumento analitico, la parte finale dell'analisi è stata svolta utilizzando Weka, una suite Java-based con interfaccia user-friendly. Inoltre, per redarre questo articolo ho impiegato il linguaggio di scrittura LaTeX.

Prima di procedere con lo svolgimento del progetto è importante capire le caratteristiche e le eventuali problematiche delle grandezze disponibili. In questa prima fase ho svolto un'analisi di carattere esplorativo per avere un quadro generale sui dati. Ho controllato la presenza di missing values ed esaminato la struttura del dataset, oltre ad alcune statistiche descrittive. Successivamente, con lo scopo di riscontrare un eventuale sbilanciamento, ho svolto un'analisi univariata calcolando frequenze assolute e percentuali per la variabile risposta e per la variabile "sex". Al fine di studiare il potere discriminante di ciascuna esplicativa rispetto alla variabile "death event", ho svolto un'analisi bivariata. Ho calcolato una tabella di contingenza tra ogni attributo categoriale e la variabile dipendente, creato un grafico boxplot per ogni variabile quantitativa e diviso le distribuzioni secondo le modalità della variabile target. In ultimo, per evidenziare un'ipotetica correlazione in ambito multivariato, ho scalato le variabili numeriche nello stesso range di valori e creato alcuni grafici di dispersione tra coppie di variabili indipendenti quantitative, in cui le osservazioni sono colorate secondo la variabile da predire.

In seguito, mi sono occupato del preprocessing del dataset. Le operazioni eseguite in questa fase di preparazione all'analisi, sono frutto dei risultati riscontrati nella precedente panoramica generale sui dati. Ho eliminato la variabile "time", in quanto il suo significato semantico non è chiaro dalla documentazione fornita dal sito. Le variabili dicotomiche, memorizzate come valori interi, sono state trasformate in factor feature. Le variabili "ejection fraction" e "serum creatine" sono state opportunatamente trasformate attraverso la funzione monotona logaritmica, in modo da risultare piu' simmetriche e caratterizzate da un minor numero di outlier.

L'obiettivo principale è fare previsione su una variabile dicotomica che indica la sopravvivenza o meno del paziente, di conseguenza si tratta di un problema di classificazione binaria nell'ambito supervisionato: si vuole predire e spiegare una variabile sulla base di altre, non cercare una struttura nei dati. Per fare ciò, mi sono servito di diversi algoritmi, provenienti da diverse aree del ML, per dimostrare la generalizzabilità dei metodi utilizzati.

Il primo approccio è stato puramente statistico; ho utilizzato un modello di regressione lineare generalizzato, in particolare un modello logit per via della natura dicotomica della variabile target. Per ridurre la varianza, come strategia di ensemble learning, ho utilizzato una random forest, facendo prima delle verifiche per settare gli iperparametri. Dato che l'errore di OOB non si riduce significativamente all'aumentare del numero di alberi della foresta, ho implementato anche un albero decisionale di classificazione. Successivamente, ho addestrato due algoritmi che lavorano bene con un numero contenuto di istanze: una support vector machine lineare e una con kernel radiale gaussiano. Per stabilire l'iperparametro C ho usato un approccio grill-based. Infine, ho usato un k-nearest-neighbor per la classificazione.

La bontà delle previsioni di ciascun algoritmo è stata valutata tramite l'area sottesa alla curva ROC, oltre ad altri indicatori basati sulla matrice di confusione: l'accuratezza, la balanced accuracy (per tenere conto dello sbilanciamento dei dati, si assegna un punteggio elevato al predittore solo se ha buoni risultati sia sulla maggior parte delle istanze di dati positivi sia sulla maggior parte delle istanze di dati negativi), la recall e la specificity.

*Comma Separated Values.

Per gli algoritmi SVM e GSVM l'intero set di dati è stato diviso randomicamente in 3 sample, il training set, il validation set e il test set, formati rispettivamente dal 60%, 30% e 10% delle unità statistiche totali. Il VS l'ho impiegato per lo studio dell'iperparametro C, mentre il TS è stato utilizzato per la stima delle misure di performance. Per l'addestramento degli altri modelli il dataset è stato suddiviso in TRS (75%) e TS (25%). All'interno dei sotto-dataset, dato il non eccessivo sbilanciamento delle classi e per rendere la divisione piú casuale e realistica, non ho mantenuto lo stesso rapporto del dataset originale tra le categorie della variabile "death event".

Per la fase di cross validation ho escluso la tecnica di bootstrap, in quanto la stima dell'accuratezza sarebbe molto ottimistica, ma in questo caso le scorrette classificazioni hanno un peso diverso: classificare una persona potenzialmente a rischio come sana è molto meno grave di classificare una persona in pericolo come in salute. Dato che la k-fold-CV con un k elevato causava problemi computazionali, ho optato per un hold-out. Per rendere le stime piú robuste ho ripetuto 100 volte la validazione per ogni algoritmo.

Alla fine di ogni validazione, a ciascun algoritmo risultano associate 100 stime per ognuno dei 5 indicatori calcolati. Come stima finale, ho calcolato la media dei 100 valori per ogni indice e, per rendere maggiormente leggibile il risultato dell'analisi, le ho collezionate in una matrice.

Sulla base di questa procedura, RF è risultato il classificatore piú efficiente sul set di dati. Su Weka, ho creato un ranking degli attributi del dataset sulla base dell'information gain. Infine, ho addestrato di nuovo la RF, caratterizzando l'algoritmo con le sole 2 variabili piú importanti secondo la classifica stilata. Per la fase di CV ho implementato una una leave one out CV, in quanto Weka, a differenza di R, non mi ha imposto limiti computazionali.

III. Results and discussion

Dall'analisi esplorativa emerge che nel dataset non ci sono valori mancanti e che tutte le variabili booleane, seppur rappresentanti di una caratteristica qualitativa individuale, sono salvate come variabili di tipo intero. Risulta esserci un leggero sbilanciamento per quanto riguarda la variabile target, infatti 203 osservazioni presentano il valore "evento morte=0" (circa il 67,9%), mentre 94 presentano il valore "evento morte=1" (circa il 32,1%). Inoltre, nella popolazione vi sono 105 donne (circa il 35,1%) e 194 uomini (circa il 64,9%). Dalle tabelle bivariate delle frequenze congiunte non risulta esserci una variabile qualitativa particolarmente discriminante circa la variabile target. Dai boxplot, rappresentati in Figura 1, si rileva che anche le variabili numeriche non presentano una buona separazione per le classi oggetto di studio. In aggiunta, ho notato che l'età varia tra i 40 e i 95 anni e che le variabili "creatinine phosphokinase" e "serum creatine" sono molto concentrate su valori bassi della distribuzione, mostrando così molti valori anomali.

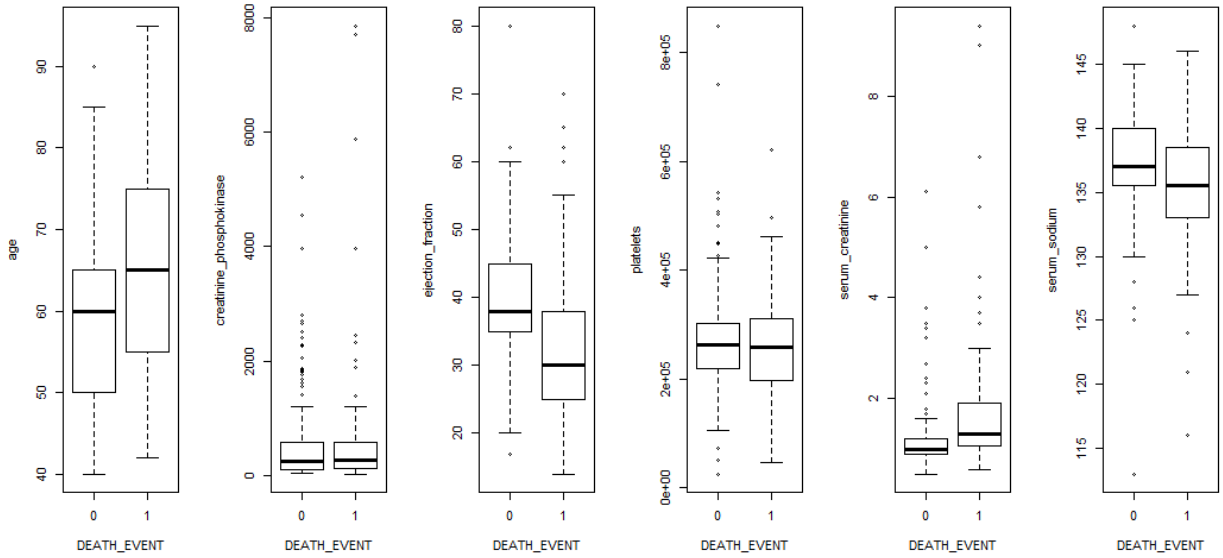


Fig. 1 Boxplot tra esplicative numeriche e risposta

In Fig. 2 sono riportati i grafici di dispersione tra coppie di covariate quantitative, in cui le unità statistiche sono colorate in funzione della variabile risposta. L'unica coppia che presenta una dipendenza significativa sembra essere "ejection fraction" e "serum creatine", in cui i data point verdi sembrano stare sopra una certa retta obliqua, mentre i data point rossi sono situati sotto (ad eccezione di alcuni, è una tendenza comportamentale). Gli altri grafici risultano essere molto confusionari, poco esplicativi ed privi di correlazione, probabilmente non tutte le esplicative sono utili a spiegare/prevedere la target feature.

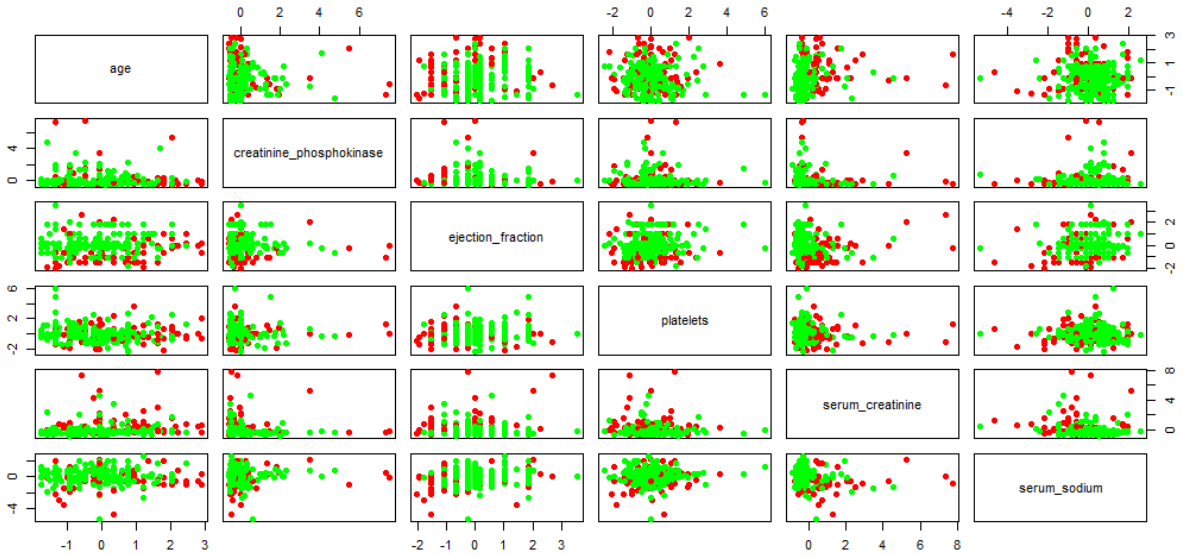


Fig. 2 Correlation plot tra esplicative numeriche

Il modello di regressione logit ha come output la tabella delle stime dei coefficienti riportata in 3. Nel parco variabili esplicative, si distinguono per significatività statistica "age", "ejection fraction" e "serumn creatinine", avendo un p-value molto basso.

```
Coefficients:
                          Estimate Std. Error z value Pr(>|z|)
(Intercept)             -1.780e+00  5.419e+00  -0.328 0.742621
age                      5.606e-02  1.547e-02   3.625 0.000289 ***
anaemia1                 6.174e-01  3.639e-01   1.697 0.089735 .
creatinine_phosphokinase 2.750e-01  1.621e-01   1.696 0.089837 .
diabetes1                2.244e-01  3.451e-01   0.650 0.515416
ejection_fraction       -8.413e-02  1.849e-02  -4.551 5.35e-06 ***
high_blood_pressure1     4.053e-01  3.610e-01   1.123 0.261597
platelets               -4.849e-07  1.960e-06  -0.247 0.804549
serum_creatinine         1.136e+00  4.238e-01   2.681 0.007330 **
serum_sodium            -1.033e-02  3.857e-02  -0.268 0.788924
sex1                    -3.303e-01  4.079e-01  -0.810 0.418076
smoking1                -1.274e-01  4.058e-01  -0.314 0.753648
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Fig. 3 Output modello logit

Per il tuning degli iperparametri della RF mi sono servito dei grafici in figura 4 e 5. Nel primo, si può osservare come l'errore decada all'aumentare del numero di alberi della foresta, per poi stabilizzarsi intorno ai 400-500 alberi. Non ha senso addestrare una foresta con un numero maggiore di alberi, in quanto ciò non comporterebbe un guadagno in termini di diminuzione dell'errore ma solo un maggior peso computazionale.

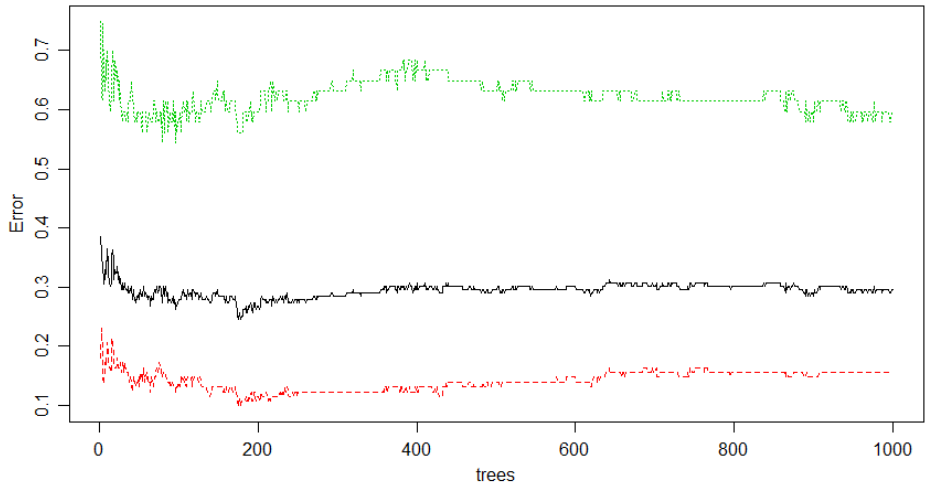


Fig. 4 Error rate RF

Il numero di feature ottimale da campionare dalla totalità delle stesse, risulta essere 3, in quanto minimizza l'errore di OOB. Ho deciso di lasciare il parametro mtry di default, poichè, nel caso di task di classificazione, corrisponde alla radice quadrata nel numero di esplicative (11 in questo caso), cioè approssimativamente proprio 3.

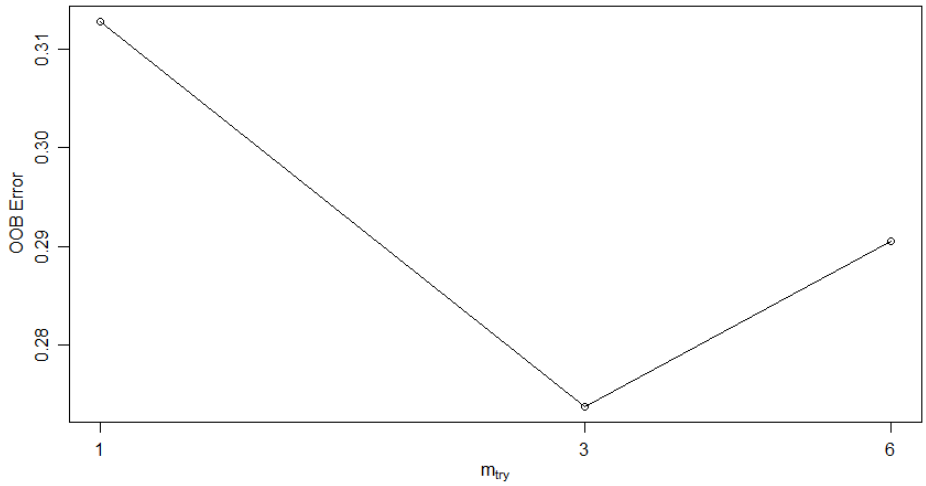


Fig. 5 Mtry RF

In Fig. 6 è riportata la matrice finale degli indicatori di performance per ciascun algoritmo.

	accuracy	balanced accuracy	recall	specificity	ROC AUC
logit model	0.7301351	0.6570833	0.8650000	0.44916667	0.6570833
random forest	0.7429730	0.6792583	0.8606000	0.49791667	0.6792583
decision tree	0.7362162	0.6914833	0.8188000	0.56416667	0.6914833
linear svm	0.7360000	0.6454882	0.8963622	0.39461409	0.6475078
gaussian svm	0.6913333	0.6113192	0.8613653	0.36127301	0.6162055
knn	0.6600000	0.5060583	0.9442000	0.06791667	0.5192583

Fig. 6 Indicatori finali bontà modelli

Tutti gli algoritmi hanno una sensitività molto elevata, cioè sono molto bravi ad identificare i pazienti sani come tali. Tuttavia, sono tutti caratterizzati da una specificità poco soddisfacente; non sono altrettanto bravi a riconoscere i pazienti potenzialmente a rischio. La RF mostra un'accuratezza piú elevata tra tutte.

Il Ranking in Figura 7, redatta grazie a Weka, mostra la lista delle variabili feature in ordine decrescente di importanza.

```
=== Attribute Selection on all input data ===

Search Method:
        Attribute ranking.

Attribute Evaluator (supervised, Class (nominal): 12 DEATH_EVENT):
        Information Gain Ranking Filter

Ranked attributes:
 0.15498969   8 serum_creatinine
 0.10179335   5 ejection_fraction
 0.04942683   1 age
 0.04285766   9 serum_sodium
 0.00449453   6 high_blood_pressure
 0.00315698   2 anaemia
 0.00011523  11 smoking
 0.00001343  10 sex
 0.00000272   4 diabetes
 0            7 platelets
 0            3 creatinine_phosphokinase
```

Fig. 7 Classifica delle feature

Le due variabili piú rilevanti sono "ejection fraction" e "serumn creatinine". Ho riscontrato una certa coerenza con i risultati precedentemente commentati, in quanto le stesse variabili erano risultate le piú discriminanti circa le categorie della variabile target e tra le piú significative statisticamente nel modello di regressione (Fig. 2 e Fig. 3).

Infine, in Figura 8 è riportata la RF addestrata su Weka con le sole variabili "ejection fraction" e "serumn creatinine".

```
=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances         224               74.9164 %
Incorrectly Classified Instances        75               25.0836 %
Kappa statistic                          0.4166
Mean absolute error                      0.2821
Root mean squared error                  0.4189
Relative absolute error                 64.4253 %
Root relative squared error             89.429  %
Total Number of Instances              299

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0,828    0,417    0,808      0,828    0,818      0,417    0,808     0,901     0
                 0,583    0,172    0,615      0,583    0,599      0,417    0,808     0,662     1
Weighted Avg.    0,749    0,338    0,746      0,749    0,747      0,417    0,808     0,824

=== Confusion Matrix ===

   a   b   <-- classified as
 168  35 |   a = 0
  40  56 |   b = 1
```

Fig. 8 RF finale

Si riscontra un leggero miglioramento nell'accuratezza, che risulta essere pari a 0.749. Se si considerasse una cifra significativa in meno, l'accuratezza sarebbe uguale a quella del RF completo precedentemente addestrato su R. A parità di accuratezza, si predilige comunque il modello piú semplice, cioè quello con meno esplicative.

IV. Conclusion

Il risultato mostra che la sopravvivenza dei pazienti con insufficienza cardiaca può essere prevista esclusivamente dalla creatinina sierica e dalla frazione di eiezione. Il livello di accuratezza ottenuto è buono, ma non sufficiente per generalizzare i risultati ottenuti. É necessario eseguire ulteriori studi di conferma prima di adottare questa procedura di apprendimento nella pratica clinica.

Come limite del presente elaborato, devo sottolineare la bassa rappresentatività del dataset dovuta al numero scarso di istanze presenti, ancor piú esiguo quando si divide il set di dati in TRS, VD e TS. Piú dati, meglio ancora se, per diversificare il dataset, provenissero da un area geografica differente, permetterebbero di ridurre sia l'errore empirico sia l'errore di generalizzazione. Inoltre, sarebbe stato interessante studiare l'eventuale relazione con ulteriori probabili fattori di rischio non raccolti sui pazienti, come ad esempio l'altezza, la percentuale di massa grassa e la presenza di malattie cardiovascolari in famiglia. Un'ulteriore criticità da evidenziare risiede nell'interpretazione personale del medico e nella soggettività degli individui; un paziente potrebbe presentare tutti i fattori di rischio ed essere in salute, mentre un altro potrebbe incorrere in problemi pur essendo apparentemente sano.

Appendix

A garanzia della riproducibilità dei risultati illustrati, si fornisce il codice R dell'analisi:

```
###Caricamento dataset e librerie###

rm(list = ls())
ls()
set.seed(1234)
library(caret)
library(class)
library(gmodels)
library(pastecs)
library(ROSE)
library(tree)
library(randomForest)
library(ModelMetrics)
library(e1071)
library(MASS)
data=read.csv("C://Users//Luciana//Desktop//Machine␣learning//
heart_failure_clinical_records_dataset.csv", sep = ",", header = TRUE)
attach(data)
fileName <- "..//heart_failure_clinical_records_dataset.csv"
targetFeature <- "death_event"
cat("fileName:␣", fileName, "\n", sep="")
cat("targetFeature:␣", targetFeature, "\n", sep="")

###Sguardo generale: prime analisi descrittive ed esplorative###

dim(data)
head(data, 10)
which(is.na(data))
str(data)
summary(data)
table(DEATH_EVENT)
100*round(prop.table(table(DEATH_EVENT)),3)
table(sex)
100*round(prop.table(table(sex)),3)
table(anaemia,DEATH_EVENT)
table(diabetes,DEATH_EVENT)
table(high_blood_pressure,DEATH_EVENT)
table(sex,DEATH_EVENT)
table(smoking,DEATH_EVENT)
par(mfrow=c(1,6))
boxplot(age~DEATH_EVENT)
boxplot(creatinine_phosphokinase~DEATH_EVENT)
boxplot(ejection_fraction~DEATH_EVENT)
boxplot(platelets~DEATH_EVENT)
boxplot(serum_creatinine~DEATH_EVENT)
boxplot(serum_sodium~DEATH_EVENT)
par(mfrow=c(1,1))
data_scaled=scale(data[,c(1,3,5,7,8,9,13)])
pairs(data_scaled[,-c(7)], col=ifelse(DEATH_EVENT==1,"red","green"), pch=20,
cex=1.4)
```

```
###Preprocessing###

data=data[,-c(12)]
data$anaemia=as.factor(data$anaemia)
data$diabetes=as.factor(data$diabetes)
data$high_blood_pressure=as.factor(data$high_blood_pressure)
data$sex=as.factor(data$sex)
data$smoking=as.factor(data$smoking)
data$DEATH_EVENT=as.factor(data$DEATH_EVENT)
data$creatinine_phosphokinase=log(data$creatinine_phosphokinase)
data$serum_creatinine=log(data$serum_creatinine)

###Corpo principale dell'analisi###

matr_metrics=matrix(ncol=5, nrow=6)

confusion_matrix = function (actual, predicted){
  conf=table(actual,predicted)
  acc=sum(diag(conf))/sum(conf)
  recall=conf[1,1]/(conf[1,1]+conf[1,2])
  specificity=conf[2,2]/(conf[2,1]+conf[2,2])
  acc_balanced=(recall+specificity)/2
  curve=roc.curve(actual,predicted)
  auc_curve=curve$auc
  return(c(acc,acc_balanced,recall,specificity,auc_curve))
}

new_matr_1=matrix(ncol=5, nrow=100)
for(i in 1:100){
  trainIndex=createDataPartition(data$DEATH_EVENT, p=0.75, list=FALSE)
  data_train=data[trainIndex,]
  data_val=data[-trainIndex,]
  reg_model_new = glm(data_train[,12] ~ ., family=binomial(link="logit"),
   data=data_train[,-12])
  data_val_pred = predict(reg_model_new, data_val[,-12], type = "response")
  data_val_pred_bin = as.numeric(data_val_pred)
  data_val_pred_bin[data_val_pred_bin>=0.5]=1
  data_val_pred_bin[data_val_pred_bin<0.5]=0
  confusion_matrix(data_val[,12],data_val_pred_bin)
  new_matr_1[i,]=confusion_matrix(data_val[,12], data_val_pred_bin)
}
summary(lin_reg_model_new)
for(i in 1:ncol(matr_metrics)){
  matr_metrics[1,i]=mean(new_matr_1[,i])
}

new_matr_2=matrix(ncol=5, nrow=100)
rf_mod=randomForest(data_train[,-12], data_train[,12] , data=data_train,
importance=TRUE, proximity=TRUE, ntree =1000)
plot(rf_mod)
t=tuneRF(data_train[,-12],data_train[,12], stepFactor = 0.5, plot=T,
 ntreeTry=300, trace = T, improve=0.05)
for(i in 1:100){
  trainIndex=createDataPartition(data$DEATH_EVENT, p=0.75, list=FALSE)
```

```
  data_train=data[trainIndex,]
  data_val=data[-trainIndex,]
  rf_mod= randomForest(data_train[,-12], data_train[,12] , data=data_train,
   importance=TRUE, proximity=TRUE, ntree =300)
  data_val_pred=predict(rf_mod, data_val[,-12], type = "response")
  new_matr_2[i,]=confusion_matrix(data_val[,12], data_val_pred)
}
for(i in 1:ncol(matr_metrics)){
  matr_metrics[2,i]=mean(new_matr_2[,i])
}

new_matr_3=matrix(ncol=5, nrow=100)
for(i in 1:100){
  trainIndex=createDataPartition(data$DEATH_EVENT, p=0.75, list=FALSE)
  data_train=data[trainIndex,]
  data_val=data[-trainIndex,]
  fit_tree=tree(DEATH_EVENT~., data=data_train, split="gini")
  fit_tree_pot=cv.tree(fit_tree, , prune.misclass, K=10)
  id.min=which.min(fit_tree$dev)
  tree_pruned_best=prune.misclass(fit_tree, best=id.min)
  plot(tree_pruned_best)
  data_val_pred=predict(tree_pruned_best, newdata=data_val, type="class")
  new_matr_3[i,]=confusion_matrix(data_val[,12], data_val_pred)
}
for(i in 1:ncol(matr_metrics)){
  matr_metrics[3,i]=mean(new_matr_3[,i])
}

new_matr_4=matrix(ncol=5, nrow=100)
for(i in 1:100){
  data_shuffled= data[sample(nrow(data)),] # shuffle the rows
  training_set_first_index= 1
  training_set_last_index= round(nrow(data_shuffled)*0.6)
  validation_set_first_index=training_set_last_index+1
  validation_set_last_index=round(nrow(data_shuffled)*0.9)
  test_set_first_index =validation_set_last_index+1
  test_set_last_index =nrow(data)
  data_train =data_shuffled[training_set_first_index:training_set_last_index,]
  data_validation = data_shuffled[validation_set_first_index:validation_set_last_index,]
  data_test =data_shuffled[test_set_first_index:test_set_last_index,]
  c_array = c(0.001, 0.01, 0.1, 1, 10)
  new=matrix(ncol=5, nrow=2)
  for(C in 1:5)
  {
    svm_mod = svm(data_train[,-c(2,4,6,10,11,12)], data_train[,12],
    cost=c_array[C], data=data_train[,-12], method = "C-classification",
    kernel = "linear")
    data_validation_PRED = predict(svm_mod, data_validation[,-c(2,4,6,10,11,12)])
    tab=table(data_validation_PRED, data_validation[,12])
    acc=sum(diag(tab))/nrow(as.matrix(data_validation_PRED))
    new[1,C]=c_array[C]
    new[2,C]=acc
  }
  param_c=new[1,which.max(new[2,])]
```

```
  svm_mod_new =svm(data_train[,-c(2,4,6,10,11,12)], data_train[,12],
  cost=param_c, data=data_train[,-12], method = "C-classification", kernel = "linear")
  data_test_PRED =predict(svm_mod_new, data_test[,-c(2,4,6,10,11,12)])
  new_matr_4[i,]=confusion_matrix(data_test[,12],data_test_PRED)
}
for(i in 1:ncol(matr_metrics)){
  matr_metrics[4,i]=mean(new_matr_4[,i])
}

new_matr_5=matrix(ncol=5, nrow=100)
for(i in 1:100){
  data_shuffled = data[sample(nrow(data)),] # shuffle the rows
  training_set_first_index = 1
  training_set_last_index = round(nrow(data_shuffled)*0.6)
  validation_set_first_index=training_set_last_index+1
  validation_set_last_index=round(nrow(data_shuffled)*0.9)
  test_set_first_index = validation_set_last_index+1
  test_set_last_index = nrow(data)
  data_train =data_shuffled[training_set_first_index:training_set_last_index,]
  data_validation= data_shuffled[validation_set_first_index:validation_set_last_index,]
  data_test =data_shuffled[test_set_first_index:test_set_last_index,]
  c_array = c(0.001, 0.01, 0.1, 1, 10)
  new=matrix(ncol=5, nrow=2)
  for(C in 1:5)
  {
    svm_mod = svm(data_train[,-c(2,4,6,10,11,12)], data_train[,12],
    cost=c_array[C], data=data_train[,-12], method = "C-classification",
    kernel = "radial")
    data_validation_PRED= predict(svm_mod, data_validation[,-c(2,4,6,10,11,12)])
    tab=table(data_validation_PRED, data_validation[,12])
    acc=sum(diag(tab))/nrow(as.matrix(data_validation_PRED))
    new[1,C]=c_array[C]
    new[2,C]=acc
  }
  param_c=new[1,which.max(new[2,])]
  svm_mod_new =svm(data_train[,-c(2,4,6,10,11,12)], data_train[,12],
  cost=param_c, data=data_train[,-12], method = "C-classification", kernel = "radial")
  data_test_PRED = predict(svm_mod_new, data_test[,-c(2,4,6,10,11,12)])
  new_matr_5[i,]=confusion_matrix(data_test[,12],data_test_PRED)
}
for(i in 1:ncol(matr_metrics)){
  matr_metrics[5,i]=mean(new_matr_5[,i])
}

new_matr_6=matrix(ncol=5, nrow=100)
for(i in 1:100){
  trainIndex=createDataPartition(data$DEATH_EVENT, p=0.75, list=FALSE)
  data_train=data[trainIndex,]
  data_val=data[-trainIndex,]
  ctrl =trainControl(method="repeatedcv",repeats = 3)
  knnFit= train(DEATH_EVENT ~ ., data = data_train, method = "knn",
   trControl = ctrl, preProcess = c("center","scale"), tuneLength = 20)
  bestK=knnFit$bestTune
  data_val_pred =knn(train = data_train, test = data_val, cl = data_train[,12],
```

```
  k=bestK)
  new_matr_6[i,]=confusion_matrix(data_val[,12],data_val_pred)
}
for(i in 1:ncol(matr_metrics)){
  matr_metrics[6,i]=mean(new_matr_6[,i])
}
colnames(matr_metrics)= c("accuracy", "balanced accuracy", "recall",
"specificity", "ROC AUC")
rownames(matr_metrics)= c("logit model", "random forest", "decision tree",
 "linear svm", "gaussian svm", "knn")
matr_metrics

###Fine dell'analisi###

detach(data)
```

References

[1] https://bmcmedinformdecismak.biomedcentral.com/articles/10.1186/s12911-020-1023-5

[2] https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0181001

[3] https://archive.ics.uci.edu/ml/datasets/Heart+failure+clinical+records